잠재변수 생성모델: VAE에서 GAN, Diffusion까지

VAE: ELBO와 실습 · GAN, Diffusion, 그리고 세 원리 비교

Machine Learning 1 · 15주차

한국과학영재학교 (KSA)

Week 15

이번 주차 개요

- **15.1 VAE** — ELBO 유도, 복원 vs KL 줄다리기, 생성 모드
- **15.2 GAN · Diffusion** — min-max 게임, 노이즈 단계별 제거, 세 원리 비교

이 주차를 마치면

ELBO를 **복원 항 + KL 정규화 항**으로 분해하고 학습 곡선에서 두 항의 **균형점**을 읽을 수 있고, GAN의 **내쉬 균형** (판별자 정확도 50%가 목표)과 **모드 붕괴**의 원인(다양성 항의 부재)을 설명할 수 있고, 세 원리를 **잠재변수 · 학습 방법 · 안정성 · 생성 속도**로 비교할 수 있다.

15.1 VAE: ELBO와 실습

EM에서 VAE로: 연속적으로, 신경망으로

- **VAE**(Variational Autoencoder, Kingma & Welling, 2013): **정확히 같은 질문** — "관측되지 않은 잠재변수가 데이터를 어떻게 만들어내는가"
- 차이: z를 유한한 클러스터 번호가 아니라 **연속적인 벡터**로
- E-step/M-step의 명시적 반복 계산을 **신경망**으로 근사

한 문장 요약

"EM의 E-step을 신경망이 맡는다" — EM/GMM에서 VAE로의 전환을 가장 짧게 요약하는 문장.

“variational”이라는 이름의 기원

- 정확한 사후 $p(z|x)$를 계산하는 대신 **함수족 안에서 최적의 근사 $q(z|x)$를 “변분(변화시켜 보며)” 구하는** 것
- “생성형 신경망”은 2013∼14년 한 번에 나온 것이 아니라 **변분 베이즈(오래된 통계 기법) × 심층학습**의 교차 지점에서 태동

방향적 그래프 모델 (원 논문 Fig. 1)

데이터 $\boldsymbol{x}$, 잠재 $\boldsymbol{z}$, 파라미터 θ. 생성 모델 $p_\theta(z)$, $p_\theta(x|z)$와 인식 모델 $q_\phi(z|x)$의 구조.

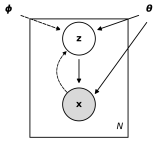


Fig. 1 재구성: 실선=생성 p_θ, 점선=근사 q_ϕ

두 전통의 합류: 오토인코더 × 베이지안

오토인코더

- 1980년대, Geoffrey Hinton 그룹의 문자·음성 복원 연구에서 발전
- 입력을 낮은 차원의 "코드"로 **압축**하고 다시 **복원**하는 순환 구조

베이지안(잠재변수) 모델

- x는 보이지 않는 원인 z가 만들어낸 관측값이라는 **생성과정**의 관점
- 15.1절 GMM이 따랐던 전통

VAE = ?

"오토인코더의 **압축·복원 구조**"에 "잠재변수 모델의 **확률적 의미**"를 입힌 것.

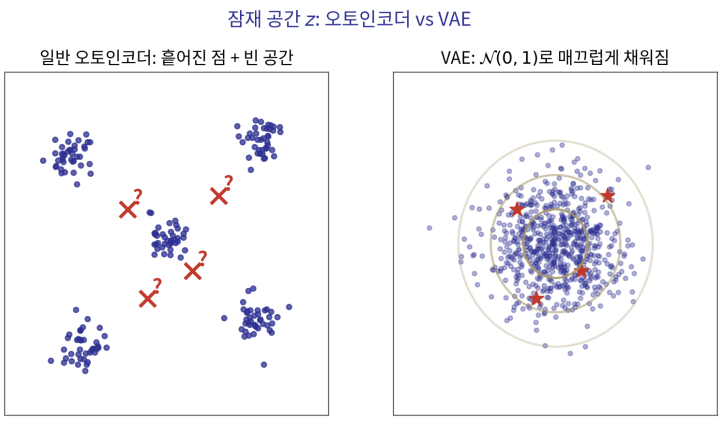
왜 분포를 내놓는가: 빈 공간 문제

- **VAE의 해법**: 인코더가 점 z 대신 **확률분포** (보통 정규분포의 평균·분산)를 내놓도록 강제
- 그 분포가 **표준정규분포**(평균 0, 분산 1)에 가깝도록 추가 제약
- **잠재 공간이 매끄럽게 채워진다**

구조

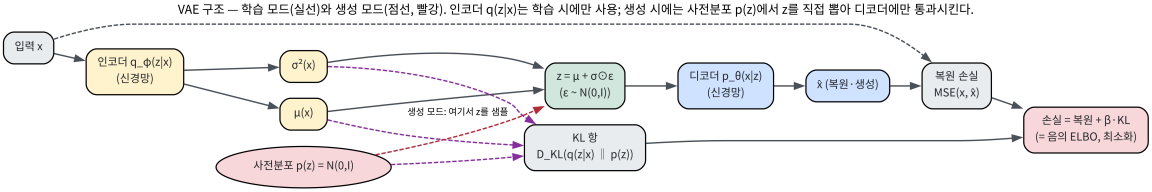
인코더 $q_\phi(z|x)$가 x로부터 z의 분포(평균·분산)를 내놓고, 그 분포에서 z를 샘플링한 뒤 디코더 $p_\theta(x|z)$가 x를 복원(생성).

그림으로: 오토인코더 vs VAE의 잠재 공간



일반 오토인코더(왼쪽)는 학습점이 여기저기 뭉쳐 있고 그 사이 **빈 공간**(빨간 $\times$)에서 뭐가 나올지 보장이 없다. VAE(오른쪽)는 모든 분포를 $\mathcal{N}(0, 1)$ 쪽으로 당겨 공간이 **매끄럽고 빈틈없이** 채워진다 — 같은 위치(빨간 별)에서도 VAE 쪽만 항상 데이터 근처.

구조도: 학습 모드 vs 생성 모드



점선 = 생성 시 흐름(인코더 미사용), 실선 = 학습 시 흐름.

학습 모드 (실선)

- x가 주어지면 인코더가 (μ, σ^2)를 내놓고, 샘플링한 z로 디코더가 x를 복원
- 인코더 분포는 사전분포 $p(z)$에 가깝도록 KL 항이 “끌어당김”

생성 모드 (점선)

- 학습 후 **인코더는 완전히 버려짐**
- 사전분포 $p(z)$에서 z를 무작위 추출 $\rightarrow$ 디코더에만 통과시켜 새 샘플

우도 기반 접근과 "적분" 문제

- **VAE는 우도 기반:** $P(x)$ **최대화**
- **적분 계산 불가능(intractable)**

GMM과의 대조 (1)

- GMM: $p(x) = \sum_k \pi_k \mathcal{N}(x; \mu_k, \sigma_k^2)$
- z가 **유한** → 유한 합, 각 항이 정규 밀도로 **닫힘** → 우도 정확히 계산 가능
- VAE: z가 **연속** → 적분으로 바뀜, 디코더가 신경망 이라 닫힌 형태 없음

GMM과의 대조 (2)

- GMM: 이산이라 사후확률(책임값) 계산 가능 → **EM** 성립
- VAE: 이 적분의 "역방향" $p(z|x)$도 계산 불가능 → **E-step 수행 불가**
- 그래서 E-step을 신경망 $q_\phi(z|x)$로 **대체**(근사)

ELBO: 계산 가능한 하한으로 우회

$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\,\|\,p(z)\big)$$

우변을 **ELBO**(Evidence Lower BOund)라 부른다. 두 항의 의미:

첫 항: 복원 항

$\mathbb{E}[\log p_\theta(x|z)]$ — 인코더가 만든 z로 디코더가 원본 x를 얼마나 잘 복원하는가 (일반 오토인코더의 복원 오차와 본질적으로 같음).

둘째 항: 정규화 항

$D_{KL}(q_\phi(z|x)\|p(z))$ — 인코더 분포가 목표 사전분포 $p(z)$ (보통 표준정규)에서 얼마나 벗어났는가 (KL divergence, Week 2.3 섀넌의 정보이론에서 소개). 이 항이 바로 잠재 공간을 매끄럽게 만드는 힘.

ELBO 유도: 엔센 부등식 한 번

$$\log p(x) = \log \int q(z)\, \frac{p(x,z)}{q(z)}\, dz = \log \mathbb{E}_{z\sim q}\left[\frac{p(x,z)}{q(z)}\right]$$

log **오목함수 엔센 부등식 적용**

$$\log p(x) \geq \mathbb{E}_q\left[\log \frac{p(x,z)}{q(z)}\right] = \underbrace{\mathbb{E}_q[\log p(x|z)]}_{\text{복원 항}} - \underbrace{D_{KL}(q\|p)}_{\text{정규화 항}}$$

마지막 등호에서 $\mathbb{E}_q[\log p(z)] - \mathbb{E}_q[\log q(z)] = -D_{KL}(q\|p)$를 썼다. **엔센 부등식 한 번이 전부**다.

갭의 정체: ELBO 최대화 = 변분 베이즈

- **ELBO 갭은 KL 발산**
- $\Rightarrow$ ELBO를 최대화하는 것은 **사후분포 근사 q를 정확히 만드는 것**과 동치

학습에서의 의미

$\log p_\theta(x) \geq$ ELBO이므로, **ELBO를 최대화하면 실제 우도의 하한도 함께 올라감** — 직접 계산 못 하는 목표를 계산 가능한 **대리(surrogate)** 목표로 바꿔치기한 것.

손으로 한 번: 1차원 장난감에서 ELBO

- **복원 항**: $\mathbb{E}_{z\sim\mathcal{N}(1,1)}[\log \mathcal{N}(x; z, 1)]$
- $\Rightarrow -\frac{1}{2}[(x-1)^2 + 1 + \log 2\pi] \approx -\frac{1}{2}[0 + 1 + 1.838] = \mathbf{-1.419}$
- **정규화 항**: $D_{KL}(\mathcal{N}(1,1)\|\mathcal{N}(0,1))$ — 정규분포 사이 KL은 닫힌 형태: $\frac{1}{2}(\sigma_q^2 + \mu_q^2 - 1 - \log \sigma_q^2) = \frac{1}{2}(1 + 1 - 1 - 0) = \mathbf{0.5}$
- **ELBO** $= -1.419 - 0.5 = \mathbf{-1.919}$

진짜 $\log p(x)$와 비교: 부등식이 실제로 성립

- **정규분포 두개의 합(적분)은 정규분포**
- $\log p(1) = -\frac{1}{4} - \frac{1}{2}\log(4\pi) \approx -\mathbf{1.516}$
- **부등식이 실제로 성립:** $-1.516 \geq -1.919$

$x = 9$로 다시 계산

복원 항 ≈ -33.419, 정규화 항 0.5 → ELBO ≈ -33.92. 진짜 $\log p(9) = \log \mathcal{N}(9; 0, 2) \approx -21.52$ — **갭이 12.40으로 벌어짐**. $q = \mathcal{N}(1, 1)$은 $x = 1$ 근처에선 "그럭저럭" 근사(갭 0.4)지만 $x = 9$엔 **완전히 부적합**(갭 12.4).

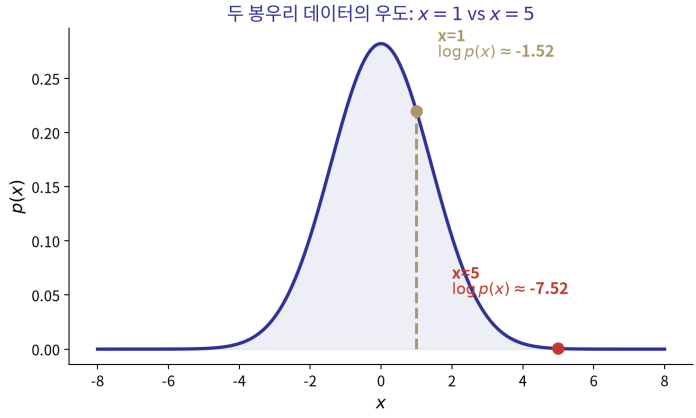
숫자가 말하는 것: 입력별 분포가 필요한 이유

- **x=5의 우도는 매우 낮음**
- $\Rightarrow$ 근사분포가 **데이터마다** 얼마나 잘 맞아야 하는지

ELBO를 최대화하는 것은 “근사분포를 잘 고르는 것”

실제 학습에서는 q를 x**마다 다른** (인코더가 만들어내는) $q_\phi(\cdot|x)$로 놓아, **복원 항이 좋아질수록 정규화 항이 나빠지는(또는 그 반대) 균형**이 학습이 끝난 지점에서 결정된다. 그 균형이 실제로 어디에 잡히는지 다음 실습에서 확인.

그림으로: $x = 1$과 $x = 5$의 우도 차이



진짜 $p(x) = \mathcal{N}(x; 0, 2)$ 위에서 $x = 1$(두 봉우리 중 하나에 가까움)과 $x = 5$(어느 봉우리에도 속하지 않음)의 높이를 직접 비교 — $x = 5$ 쪽이 **훨씬 낮다**. 근사분포 $q(z|x)$가 이만큼 다른 x들을 전부 감당하려면 x**마다 다른 분포**가 필요.

VAE 손실함수: ELBO의 코드

```python
def vae_loss(x, x_reconstructed, mu, log_var):
    # 복원손실여기서는  ( 로MSE 근사)
    recon_loss = sum((x[i] - x_reconstructed[i]) ** 2 for i in range(len(x)))
    # KL divergence: 정규분포 q(mu, sigma^2)와 표준정규 N(0,1)의 닫힌형태
    kl_loss = -0.5 * sum(1 + log_var[i] - mu[i]**2 - math.exp(log_var[i])
                         for i in range(len(mu)))
    return recon_loss + kl_loss  # ELBO 최대화 = 음의 ELBO 최소화
```

정규분포 $\mathcal{N}(\mu, \sigma^2)$와 표준정규 $\mathcal{N}(0,1)$ 사이의 KL은 닫힌 형태:

$$D_{KL}\big(\mathcal{N}(\mu,\sigma^2)\,\|\,\mathcal{N}(0,1)\big) = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right)$$

Reparameterization Trick

- **Reparameterization Trick** (Rezende et al., 2014):

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 1)$$

왜 꼭 필요한가 (Week 9 연결)

역전파는 연쇄법칙으로 미분 가능한 연산 사슬을 거슬러 올라가는 것 — "확률분포에서 무작위 추출" 연산은 미분이 정의되지 않는다. 이 트릭이 없으면 VAE의 인코더를 학습할 수 없다.

실습: 두 봉우리 데이터로 작은 VAE

15.1절 GMM 예제와 같은 모양(1 근처·9 근처 두 봉우리)의 1차원 데이터로 실제 VAE를 학습:

```
import torch, torch.nn as nn

class VAE(nn.Module):
    def __init__(self, latent_dim=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(1, 16), nn.ReLU())
        self.mu = nn.Linear(16, latent_dim)
        self.logvar = nn.Linear(16, latent_dim)
        self.dec = nn.Sequential(nn.Linear(latent_dim, 16), nn.ReLU(), nn.Linear(16, 1))
    def forward(self, x):
        h = self.enc(x)
        mu, logvar = self.mu(h), self.logvar(h)
        z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)  # reparameterization trick
        return self.dec(z), mu, logvar
```

학습 곡선: 두 항의 줄다리기

에폭	loss (복원 손실, KL)
0	57.381 (54.392, 2.989)
100	3.051 (0.852, 2.199)
499	2.063 (0.482, 1.581)

- **손실 57.4 → 2.06 감소**
- **복원 손실 소폭 상승 구간**

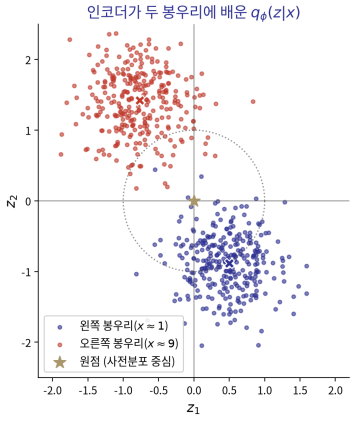
인코더가 두 봉우리에 배운 분포

- 왼쪽 봉우리(≈ 1)의 점들: 평균 $(\mu_1, \mu_2) \approx (0.5, -0.88)$
- 오른쪽(≈ 9): $(-0.77, 1.42)$ 근처로 **분리**
- 분산은 아주 작음 ($\log \sigma^2 \approx -1.7$ 이하)

GMM과의 연결

인코더가 "왼쪽 점"과 "오른쪽 점"을 잠재 공간에서 **다른 위치에, 좁은 분포로** 배치 — GMM의 이산적 클러스터 할당에 해당하는 **연속적(soft) 버전**.

그림으로: 인코더가 배운 두 개의 좁은 분포



왼쪽 봉우리($x \approx 1$)와 오른쪽 봉우리($x \approx 9$)가 잠재 공간 (z_1, z_2)에서 **서로 다른 위치에, 좁은 분산으로** 분리되고, 원점(사전분포 $\mathcal{N}(0,1)$의 중심)을 사이에 두고 대략 대칭 — GMM의 이산적 클러스터 할당에 해당하는 **연속적(soft) 버전**.

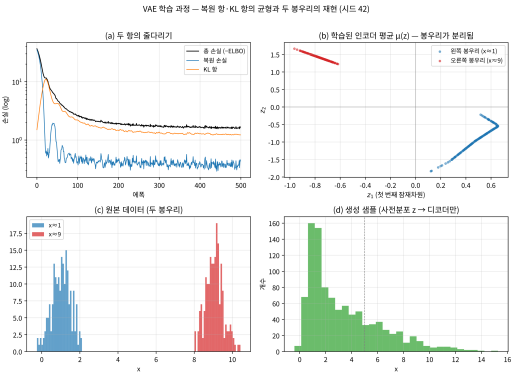
생성 모드: 원본을 전혀 보지 않고

원본 데이터 없이 표준정규 z 추출

생성된 샘플: 평균 4.83, 표준편차 3.84
1(왼쪽 봉우리) 근처 비율: 59.1%, 9(오른쪽 봉우리) 근처 비율: 40.9%

- 원본이 두 봉우리에 대략 절반씩인데, 디코더가 잠재 공간의 표준정규 z만으로 그 두 봉우리를 **대략 6:4로 재현**

학습 과정 2×2 시각화



(a) 손실 곡선 (log) (b) 인코더가 매긴 $\mu(z)$ (c) 원본 (d) 생성 샘플

- **(a)**: KL 항이 2.2에서 1.2로 **큰 폭으로 떨어짐**, 복원 손실은 0.43 부근에서 **거의 제자리** — 두 항이 서로 "견제"하며 균형으로 수렴
- **(b)**: GMM의 이산적 클러스터 할당의 **연속적(soft) 버전** — 각 점에 매긴 평균이 "왼쪽 $\approx (0.5, -0.9)$", "오른쪽 $\approx (-0.8, 1.4)$" 근처의 **좁은 군집**으로 모임
- **(c)(d)**: 생성 샘플(1000개)이 두 봉우리를 히스토그램으로 대략 재현

β-VAE: 두 항의 균형점을 "학습률처럼"

β-**VAE:** $\mathcal{L} = \mathbb{E}[\log p(x|z)] - \beta\, D_{KL}$

β	복원 손실	KL	생성 샘플 표준편차
0	0.016	19.1	0.58
0.1	0.068	5.61	0.91
0.5	0.232	1.70	1.67
1.0 (기본)	0.378	1.22	2.05
2.0	0.546	0.90	2.42

β는 복원 품질 vs 잠재 공간 매끄러움의 균형

자주 하는 실수 ①②: KL 항 계산

① 복원 항 기대값에 σ^2 항을 빼먹는 것

$\mathbb{E}[\log p(x|z)]$를 계산할 때 $(x-\mu)^2$만 쓰고 분산 σ_q^2 항을 빼먹으면 복원 항이 -0.919로 **과소평가**(정답 -1.419). $z = \mu + \sigma\epsilon$일 때 $\mathbb{E}[(x-z)^2] = (x-\mu)^2 + \sigma^2$ — "평균으로 대체"한 것만으로 오차가 사라지지 않음. σ^2가 큰(인코더가 불확실한) 입력에서 손실값이 실제보다 크게 나타나는(= ELBO 과소평가) 방향으로 오류를 만들어, "정규화 항을 줄이는데 왜 손실이 안 줄지"를 진단할 때 혼란의 원인.

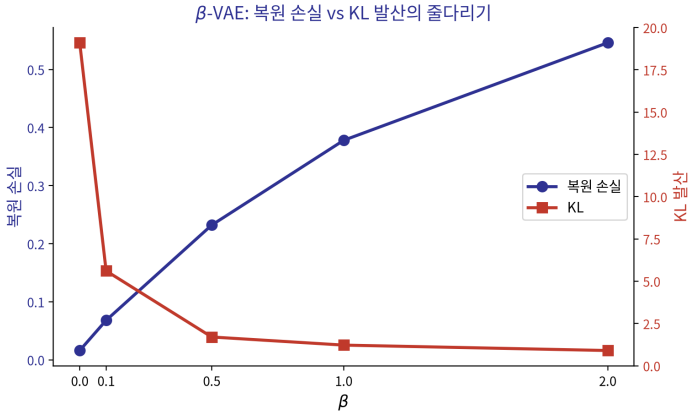
② KL 항을 "분산만" 계산하는 것

인코더가 (μ, σ^2)를 함께 내놓는데, KL 항에 μ^2(위치) 항을 빼먹고 $\sigma^2 - 1 - \log\sigma^2$만 쓰면, μ**에 대한 그래디언트가 0** — 인코더의 평균 출력 위치에 대해 정규화 항이 아무 작용도 하지 않아 "분산만 조절된다"는, 학습이 이상하게 멈추는 원인. $\frac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2)$의 μ^2 항이 **인코더의 위치(μ)를 원점 방향으로 끌어당기는 힘**임을 꼭 기억.

자주 하는 실수 ③④⑤

- **③ β를 너무 크게 올려 "정규화만" 보는 학습** — β-VAE에서 β를 극단적으로 올리면 KL이 0에 가까워지고 복원 손실은 커짐 — "인코더가 데이터를 전혀 담지 않고 표준정규분포를 그대로 뱉는" 방향으로 **치팅**할 유인이 강해짐. $\beta = 2$ 이미 복원 손실 0.55(기본 1.0 대비 1.45배). "KL을 많이 줄이면 잠재 공간이 더 매끄러우니 좋다"는 생각은 β-VAE의 **균형을 무너뜨리는** 것.

그림으로: β가 키우는 트레이드오프



이전 슬라이드 표(0, 0.1, 0.5, 1.0, 2.0)를 그대로 그래프로 — β가 커질수록 **KL(파랑 → 빨강 곡선)은 떨어지고 복원 손실은 올라간다**. β를 극단적으로 키우면 KL이 0에 가까워지며 "치팅" 방향(정규화만 보는 학습)으로 무너지는 이유가 이 곡선에 그대로 보인다.

확인 문제 (15.1)

1. **ELBO 부등식의 갭.** $\log p(x) - \text{ELBO} = D_{KL}(q \| p(z|x))$임을 유도한 뒤, $q = p(z|x)$일 때만 갭이 0이 된다는 것을 한 문장으로 설명.
2. **정규분포 합 = 정규분포.** $\mathcal{N}(x; z, 1) \cdot \mathcal{N}(z; 0, 1)$에 대해 z로 적분 하면 $\mathcal{N}(x; 0, 2)$가 된다는 것을, "두 독립 정규분포의 합은 정규분포"라는 사실로부터 한 문장으로 설명.
3. **손계산 재현.** $x = 9$ 예제에서 복원 항이 ≈ -33.4가 되는 것을 직접 계산 (힌트: $(x-\mu)^2 = 64$, $\sigma_q^2 = 1$, $\log 2\pi \approx 1.838$ 대입).

핵심 요약

ELBO = **복원 항** $-$ **KL 정규화 항**. **ELBO 최대화 = 사후분포 근사 q를 정확히 만드는 것 (변분 베이즈)**. μ^2 항은 **위치를 원점 쪽으로 끌어당기는 힘**, σ^2 항은 **너비를 제어**.

자주 묻는 질문: 두 항 중 하나만 최적화하면?

- **복원 항만**(KL 없이): 일반 오토인코더와 같아져 잠재 공간이 매끄럽지 않아 샘플링 품질이 나빠짐 — 실습에서 KL 항이 없었다면 생성 샘플이 두 봉우리를 재현하지 못했을 가능성 큼
- **KL 항만**(복원 없이): 인코더가 아무 정보도 담지 않고 그냥 표준정규분포를 그대로 뱉는 방향으로 "치팅" — 복원이 전혀 안 되는 무의미한 모델
- ⇒ **두 항의 균형이 VAE의 핵심**

자주 묻는 질문: 리파라미터화 · 흐릿함

Reparameterization trick이 왜 꼭 필요한가?

Week 9: 역전파는 연쇄법칙으로 미분 가능한 연산 사슬을 거슬러 올라가는 것. “확률분포에서 무작위 추출” 연산은 미분이 정의 안 됨. $z = \mu + \sigma \odot \epsilon$으로 다시 쓰면 무작위성은 ϵ(입력과 무관한 상수)에만, μ, σ로 가는 경로는 전부 미분 가능(곱셈·덧셈) → 역전파가 인코더 파라미터까지 정상적으로 흐름.

KL 항이 “흐릿함”의 원인이라는 말은?

복원 항은 **기대값** $\mathbb{E}_{z\sim q}[\log p(x|z)]$를 최적화 — 인코더 분포의 **모든** z로 복원했을 때의 **평균**이 좋아야 함. 분포가 넓을수록(σ^2 클수록) 복원된 x 후보가 넓게 퍼져 “모든 z의 평균”이 흐려짐 → VAE 생성 이미지의 **흐릿함(blurry)** 근원. β를 크게 하면 더 강해짐 (정보를 더 압축하니까). GAN이 “한 개 z → 한 개 x”를 **결정적으로** 학습(흐릿하지 않은 선명함)하는 이유도 결국 “기대값 최적화 vs 결정적 복원”의 차이.

자주 묻는 질문: EM/GMM과의 관계 · 차원 선택

VAE와 15.1절 EM/GMM의 관계?

GMM: z가 **이산**(클러스터 번호)이라 E-step(책임값)이 닫혀 계산 가능 → EM 성립. VAE: z가 **연속**이라 E-step 자체가 계산 불가능 → **E-step을 신경망 $q_\phi(z|x)$로 대체**(근사), M-step을 **역전파**로 수행. "EM의 E-step을 신경망이 대신한다". M-step도 "잠재변수별 파라미터 가중평균"이 아니라 "전체 파라미터를 ELBO의 그래디언트로 갱신"으로 바뀜.

`latent_dim` $= 2$가 왜 2차원인가?

학습 대상이 아니라 하이퍼파라미터. 두 봉우리 데이터: "두 가지 원인"이 있어 1차원으로 충분, 2차원이면 두 봉우리가 원점 중심의 **대칭** 위치로 자연스럽게 배치. 이미지 데이터: 20∼200 차원이 흔함 — 너무 작으면 복원 품질 떨어짐, 너무 크면 "사용하지 않는 차원"으로 가득 차 KL이 커짐.

15.2 GAN, Diffusion, 그리고 세 원리 비교

15.1의 다음: 우도 기반과 확연히 다른 두 방식

- 이번 절: 우도 기반과 **확연히 다른** 두 방식
- **GAN**: 두 신경망의 **경쟁 게임**으로 학습
- **Diffusion**: 노이즈를 **한 단계씩 되돌리는 회귀 문제**로 학습

GAN: 생성자와 판별자의 min-max 게임

- 2014년, **이언 굿펠로우(Ian Goodfellow)**: “진짜 같은 가짜를 만드는 모델 학습” 대신 **두 신경망이 서로 경쟁**하게 하면?
- **위조범**(생성자 G): 가짜를 만듦, **감정사**(판별자 D): 진짜/가짜를 구별
- VAE의 “우도 최대화”라는 **단일** 목표함수와 달리, GAN은 두 네트워크가 **서로 반대되는** 목표를 갖고 동시에 학습

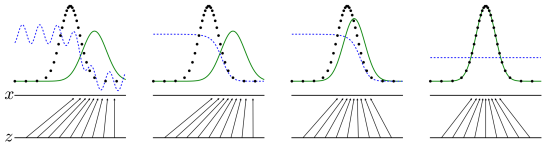


Fig. 1: 1차원 toy 예제 — 검은 점=실제 데이터 분포, 초록 선=생성자 분포, 파랑 점선=판별자 출력. (a)→(d)로 갈수록 생성자가 데이터 분포에 수렴, 판별자는 $D=1/2$로 구별 불가능해짐.

역사적 일화

2015년 GAN 논문이 ICLR 워크숍에서 거절, 심사평 “**average quality**”. 2016년 ACM 튜링상 수상 연설에서 굿펠로우가 직접 소개한 아이러니.



Ian Goodfellow — GAN 고안 (2014)



Yoshua Bengio — 지도교수, 2018 ACM 튜링상
(Bengio, Hinton, LeCun 공동 수상)

생성자·판별자와 목적함수

- **생성자**(Generator) G: 무작위 노이즈 $z \sim p(z)$를 입력받아 가짜 데이터 $G(z)$를 만듦
- **판별자**(Discriminator) D: 데이터를 입력받아, 그것이 **진짜일 확률** $D(x) \in [0,1]$을 출력 (Week 2 **로지스틱회귀**와 정확히 같은 형태)

목적함수 (min-max game):

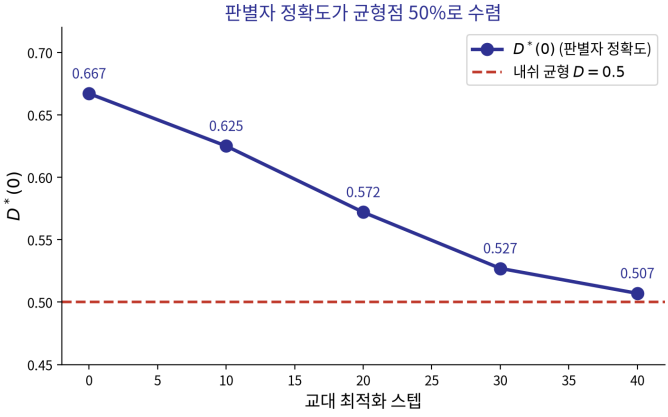
$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$$

판별자 손실	$-\big(\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]\big)$
생성자 손실	$-\mathbb{E}[\log D(G(z))]$ (D_fake를 1에 가깝게)

왜 “균형점”을 논해야 하는가

- 게임이론에서 이런 안정 상태 = **내쉬 균형**(Nash equilibrium) — **어느 쪽도 혼자서 전략을 바꿔서 더 나아질 수 없는 상태**
- **내쉬 균형: D(x)=0.5, G 분포=실제 데이터**

그림으로: 판별자 정확도가 50%로 수렴



40스텝 교대 최적화 궤적(뒤 슬라이드 표 데이터) — $D^*(0)$이 0.667에서 **0.5(내쉬 균형)로 꾸준히 접근**.
실전 판독의 기준선: 판별자 정확도 50%는 실패가 아니라 **목표 운영점**.

1차원 장난감: 파라미터가 딱 1개인 GAN

- 데이터 $x \sim \mathcal{N}(0, 1)$, 생성자 분포 $p_G = \mathcal{N}(0, s^2)$
- **표준편차 s 하나로** 가짜 분포의 "너비"만 바꿀 수 있는 생성자

판별자는 로지스틱회귀

D는 x 위의 로지스틱회귀(Week 2.3). G를 고정하고 D를 **완전히** 최적화하면(미니배치가 아니라 전체 데이터에서 MLE로) 해는 **닫힌 형태**:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

놀라운 닫힌 형태: JSD로 게임의 값

$$V(D^*, G) = 2\,\mathrm{JSD}(p_{\text{data}} \| p_G) - \log 4$$

JSD (Jensen-Shannon Divergence)

KL 발산의 "**양방향 평균**" 버전: $\mathrm{JSD}(p\|q) = \frac{1}{2}D_{KL}(p\|m) + \frac{1}{2}D_{KL}(q\|m), m = \frac{1}{2}(p+q)$. KL은 p, q 순서가 바뀌면 값이 달라지는 **비대칭** 발산인데, JSD는 두 방향의 평균이라 **대칭**.

- JSD ≥ 0이고 두 분포가 **정확히 같을 때만** 0 → V의 최솟값은 $p_G = p_{\text{data}}$에서의 $-\log 4$
- JSD $\leq \log 2$ → 게임의 값은 항상 $[-\log 4,\ 0)$ 안에

“게임 값을 낮추라” = “JSD를 줄이라”

생성자 관점의 명령

“게임 값을 낮추라”는 것은 **“가짜와 데이터 분포의 JSD를 줄이라”**는 것과 **정확히 같은 명령** — 두 분포를 일치시키는 것, 즉 내쉬 균형 $s^* = 1$으로 수렴.

1. G를 고정하고 D를 최적화 (장난감에서는 닫힌 형태 D^*라 바로 점프)
2. D^*를 고정하고 G의 파라미터 s를 그라디언트 스텝으로 한 스텝 갱신

EM과의 결정적 차이

EM

- E-step이 **정확한** 사후확률을 채워 넣음
- “우도가 매 반복 절대 줄지 않는다”는 보장이 **증명 가능**

GAN 루프

- (1)이 일반적으로 **완전 최적화가 아닌 근사**
- 매 D 갱신마다 G가 내려다보는 **경사면 자체가 바뀌고**
- 게임 값에 **단조 감소 보장이 없음**

이 장난감이 운 좋은 이유

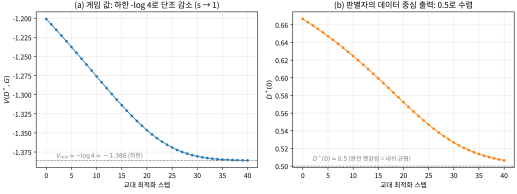
게임 값이 s의 매끄러운 함수로 **유일하게** 최소가 $s = 1$ 에 있어 **단조하게** 수렴. 실전 GAN의 “나쁜 소식들”(등락, 발산, 모드 붕괴)이 아직 안 보이는 이유 = 이 장난감에 **국소 균형이 딱 하나**뿐. 여러 개의 국소 균형이 존재하려면, 데이터 분포의 “여러 모드 중 일부만 덮는” 가짜 분포도 균형 후보가 되는 **고차원 세계**가 필요.

40스텝 교대 최적화 궤적

$s_0 = 2.0$, 학습률 $\eta = 0.15$로 40스텝 (수반 노트북 2절):

스텝	s	$V(D^*, G)$	$D^*(0)$
0	2.000	-1.201	0.667
10	1.664	-1.276	0.625
20	1.339	-1.346	0.572
30	1.113	-1.381	0.527
40	1.026	-1.386	0.507
∞	1.000	$-\log 4 \approx -1.386$	0.500

- **(i)** 균형점의 게임 값은 $-\log 4 \approx -1.386$ 인데 **0이 아님** — "생성자 손실이 0에 가까워지면 성공"은 **틀린 읽기**. 성공 신호 = V가 더 이상 줄지 않고 $-\log 4$ 근방에서 **안정**
- **(ii)** $D^*(0)$(판별자가 데이터 중심 $x = 0$을 "진짜"로 판정하는 확률)이 0.667에서 0.5로 **점점 구별력을 잃는** 궤적 — "내쉬 균형 = 판별자가 완전히 헷갈리는 점"의 실행
- **(iii)** V는 줄지만 **0으로 가지 않음** — 게임 값 하한 $-\log 4$. 양쪽이 완벽히 헷갈려도 손실은 양쪽 로그 항이 $\log 0.5 = -\log 2$씩 기여해 합계 $-\log 4$



왼쪽: $V(D^*, G)$ 궤적, 오른쪽: $D^*(0)$ 수렴

장난감의 한계: 평균을 못 움직임

- 장난감 생성자는 평균을 못 움직임

실전 학습: 교대 최적화의 함정

- (1) 판별자 갱신: G 고정, D만 한 스텝
- (2) 생성자 갱신: D 고정, G만 한 스텝 — G_loss의 그래디언트가 D를 "지나" G의 파라미터로 흐름

EM과의 차이 (다시)

EM: E-step이 **정확한** 사후확률을 채워넣음 → "우도가 단조증가"가 **증명 가능**. GAN: (1)이 D를 완전 최적화하지도, (2)가 G를 완전 최적화하지도 못하는 **근사적** 게임 최적화 → 학습 곡선이 "꾸준히 하강하는 우도"처럼 보이지 않고, 오히려 **등락을 거듭**.

불안정성의 세 가지 양상

- **학습 속도 밸런스가 중요**

실수 1: D 정확도 50%를 “실패”로 읽기

거꾸다

이 게임에서 D의 정확도 50%는 **가장 이상적인 운영점** (내쉬 균형).

- 학습 중 D의 정확도가 **50% 부근**에서 머물면 “양쪽이 서로를 잘 따라가고 있다”는 **신호**
- 반대로 D의 정확도가 **90%+**로 치솟으면 G**가 이미 학습에서 이탈한 전형적 실패 신호** — “D가 잘 learned하고 있다”가 아니라 **“G가 D를 못 따라가고 있다”**는 뜻

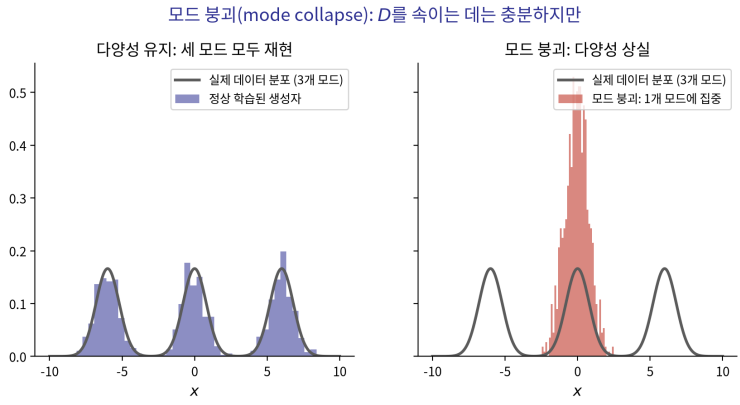
모드 붕괴: “성공”이 “실패”가 되는 지점

- **모드 붕괴: 다양성 상실**
- VAE: 이론적으로 **안정적**이지만 생성 품질이 다소 **흐릿**(blurry) 경향
- GAN: **선명한** 결과이지만 **학습이 까다로움**

1차원 장난감이 이미 메커니즘을 보여줌

기본 GAN 목적함수에는 **“다양성을 유지하라는 항”이 없음**. 생성자 손실 $-\log D(G(z))$는 오직 “이 가짜가 D를 얼마나 속였는가”에 의존. 데이터 분포가 세 개의 모드(예: 세 종류의 동물)를 갖고 있어도, **하나의** 모드만 집중적으로 만들어도 D를 잘 속일 수 있다면 — 고차원 공간에서 D가 “세 모드 모두를 동시에 감시”할 수 없는 한 — 그 하나에 집중하는 것이 손실을 줄이는 **가장 확실한** 전략.

그림으로: 모드 붕괴가 실제로 이렇게 생겼다



실제 데이터가 세 개의 모드를 가질 때, 정상 학습된 생성자(왼쪽)는 세 모드를 **모두** 재현하지만, 모드 붕괴(오른쪽)는 그중 **하나에만 집중** — D를 속이는 데는 충분해도 다양성을 잃는다.

VAE에는 이 항이 있다: 핵심 구조적 차이

VAE (15.1절)

- KL 항 $D_{KL}(q_\phi(z|x)\|p(z))$이 “인코더 출력 분포가 표준정규분포에 가깝게 유지”를 **목적함수 안에 명시적으로** 요구
- 잠재 공간 **전체**가 쓰이는 구조적 압력
- “잠재 공간의 일부만 쓰는” 퇴화 = **ELBO를 직접적으로 낮추는 행위**

GAN

- 내쉬 균형 $p_G = p_{\text{data}}$ (모든 모드를 덮는 분포)가 “완전한 균형”
- 실제 학습이 풀고 있는 **근사적** 게임에서는 “일부 모드를 덮고 D를 속이는” 분포도 **국소적으로 균형처럼** 작용
- 이 대응 **항이 없음**

이것이 VAE/GAN 비교에서 반복되는 핵심 구조적 차이.

2014년 이후 GAN 계열: “안정화”가 3년의 주제

- **WGAN: 워터스톤 거리로 안정화**
- **CycleGAN: 레이블 없는 이미지 변환**
- **StyleGAN: 스타일 수준별 제어**
- **BigGAN: 대규모 조건부 생성**



StyleGAN류 생성기가 만든 얼굴 — 실존 인물이 **아님** (This Person Does Not Exist 예시)

Diffusion: 정방향/역방향 과정

정방향 과정 (forward process)

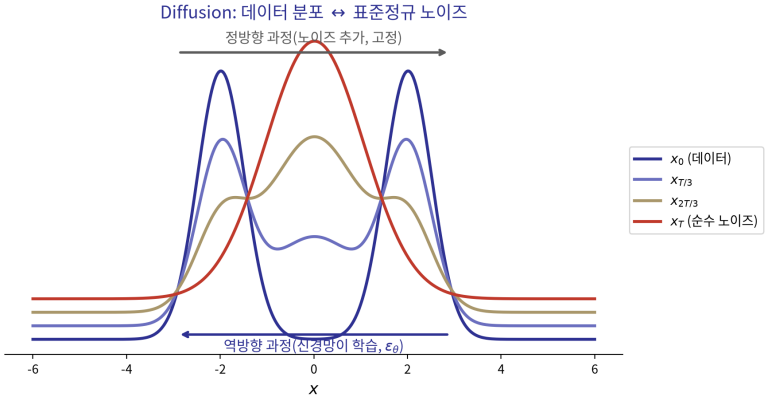
진짜 이미지 x_0에 **아주 작은 노이즈를 T번 반복**해서 추가해, 결국 x_T가 **순수한 무작위 노이즈**가 됨.
이 과정은 **고정된 절차이지 학습 대상이 아님**.

역방향 과정 (reverse process)

신경망이 **"한 단계 전 노이즈가 어땠는지"**를 예측하도록 학습. 학습 후, 순수 노이즈 x_T에서 시작해 이 역방향 단계를 T번 반복 → **새로운 이미지**.

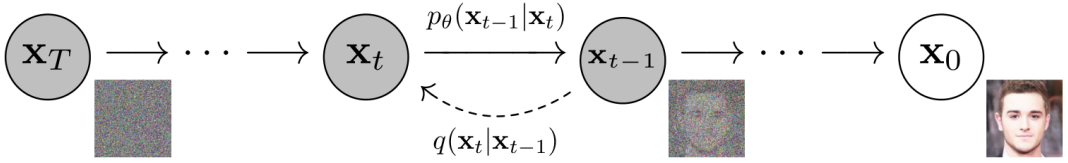
- **Diffusion은 작은 단계 T개로 쪼갬**

그림으로: 데이터 분포 ↔ 노이즈



정방향(위쪽 화살표, 고정)은 두 봉우리 데이터 분포를 표준정규 $\mathcal{N}(0,1)$까지 점점 뭉갠다. 역방향(아래쪽 화살표, ϵ_θ가 학습)은 그 과정을 거슬러 **노이즈에서 데이터 분포를 되살린다**.

그림으로: Diffusion의 마르코프 체인



x_T(순수 노이즈)에서 x_0(완성된 이미지)까지 — 실선 화살표 $p_\theta(x_{t-1}|x_t)$가 학습되는 **역방향(생성)** 과정, 점선 화살표 $q(x_t|x_{t-1})$가 고정된 **정방향(노이즈 추가)** 과정.

정방향은 왜 “학습 대상”이 아닌가

- 정방향 과정은 **고정된, 알려진** 절차:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I)$$

- **역방향 추측이 어려운 이유**

학습 대상은 하나뿐

역방향 모델 $p_\theta(x_{t-1} \mid x_t)$.

“노이즈 예측” 회귀 문제 (DDPM)

DDPM: 신경망 $\epsilon_\theta(x_t, t)$로 노이즈 ϵ_t 예측

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$$

이 식이 MSE 회귀 손실이라는 점에 주목

이 장에서 본 모든 방식(EM의 M-step, VAE의 ELBO 역전파)과 달리, GAN의 min-max 게임도 아니고, 우도의 적분 근사도 아닌, **가장 보통의 단일 회귀 최적화**. “안정적으로 학습된다”의 **정체**가 바로 이것.

ϵ 예측은 스코어 함수 $\nabla_x \log p_t(x)$ 추정과 동치

역방향 단계(DDPM)와 “다시 노이즈 추가”

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon_\theta(x_t, t)\right) + \sigma_t\, z, \quad z \sim \mathcal{N}(0, I)$$

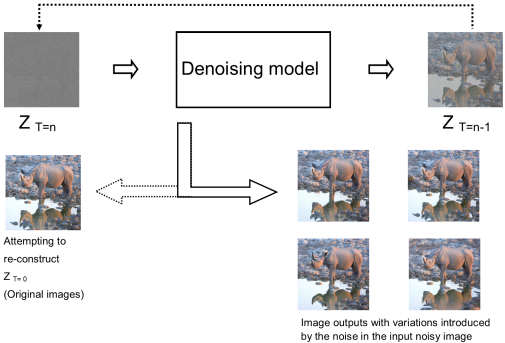
- “ϵ_θ만큼 **빼고**(조금 깨끗해지고), $\sigma_t z$만큼 **다시 더한다**(다시 노이즈를 조금)”

자주 하는 실수 3

“denoise = 계속 깨끗해지는 과정”이라는 직관과 달리, DDPM의 역방향 단계에는 $+\sigma_t z$(**다시 노이즈 추가**)가 있음. 이 항을 빼면 역방향 과정이 더 이상 $p(x_{t-1} \mid x_t)$ 가 아니라 **“점 추정”**으로 바뀌고, 생성 품질이 크게 떨어짐. $x_{t-1} \mid x_t$가 **분포**라는 점을 잊지 말 것.

역사: Sohl-Dickstein 2015 $\rightarrow$ DDPM 2020

- **DDPM: 노이즈 예측, MSE 손실, 실용적 스케줄**
- 같은 해 **DDIM** (Song et al.): 역방향 과정을 **거의 결정적으로** 만들어 T를 **1000에서 약 50** 단계로 줄임
- **Stable Diffusion: VAE 조합, 잠재 공간 Diffu**



노이즈 → 이미지: 실제 Denoising Diffusion 과정 예시



Stable Diffusion(2022) 생성 결과 예시

GAN vs Diffusion: “학습할 것” 한 줄

	GAN	Diffusion
학습할 네트워크	2개 (G, D)	1개 (ϵ_θ)
학습 목표	게임의 **균형**(닫힌 형태 없음)	$\|\epsilon - \epsilon_\theta\|^2$ (**MSE**)
샘플링 시 순전파	**1회**	T회 (DDIM으로 $\sim$50회)

이미지 밖으로: Waymo SceneDiffuser

- SceneDiffuser: 자연어 지시로 3D 교통 장면 생성 및 시뮬
- 장면 수준 Diffusion 사전분포로 스페이오-템포럴 예측
- 생성 모델이 안전 검증 도구로 쓰이는 대표 사례



Waymo 자율주행차 — SceneDiffuser(2024)를 만든 실제 회사

세 원리 비교

	VAE (우도 기반)	GAN (적대적)	Diffusion (스코어 기반
잠재변수	연속 **벡터**	**없음**(노이즈를 직접 변환)	연속(노이즈 **단계**)
학습 방법	ELBO **최대화**(역전파)	**min-max 게임**의 균형	각 단계 **노이즈 예측** 오 최소화
학습 안정성	**비교적 안정적**	**불안정하기 쉬움**	**안정적**
생성 속도	**빠름**(한 번에)	**빠름**(한 번에)	**느림**(반복 필요)

잠재변수 행 다시 읽기

- **VAE**의 z = **연속 벡터** — 가능치가 무한해서 E-step을 신경망으로 **근사**해야 함 (15.1절)
- **GAN**의 z = **분포가 아니라 그냥 랜덤 입력** — "데이터를 설명하는 잠재변수"의 역할을 **하지 않고**, 생성자가 입력으로 받는 **다양성의 원천**일 뿐 (z의 분포를 학습하지도, $z \mid x$를 추론하지도 않음)
- **Diffusion**의 "잠재변수" = **중간 상태** x_t — 연속적이지만 **단계** t**가 이산적으로** T**개인**, "점진적으로 오염된 데이터" 그 자체

학습 방법 행 다시 읽기: 안정성의 원인

- **VAE**: ELBO(단일 목적함수)를 역전파로 최대화하는 **보통의** 최적화 → **안정**
- **Diffusion**: MSE 회귀라 **가장 보통의** 최적화 → **안정**
- **GAN은 안정성 근거가 없음**

생성 속도 행: “안정성의 대가”

- 가장 **안정한** Diffusion은 생성에 T**회 반복** 필요 (DDIM으로 줄여도 수십 회)
- 가장 **빠른** GAN/VAE는 **한 번의** 순전파로 생성

장 전체를 관통하는 트레이드오프

“**데이터를 설명할 수 있다**(안정성, 평가 가능)” vs “**빨리 만들 수 있다**(속도)”

실전 시스템은 “하나씩 고르는 것”이 아니라 **조합**

Stable Diffusion = **VAE**(잠재 공간) $\times$ **Diffusion**(노이즈 제거). SceneDiffuser = **Diffusion** 원리를 **시간 축의 장면**으로 확장. $\Rightarrow$ “세 원리를 나란히 보고 나서야, “관측되지 않은 구조가 데이터를 만들어낸다”는 하나의 질문이 얼마나 다양한 방식으로 풀릴 수 있는지”가 보임.

자주 하는 실수 2·4: GAN/Diffusion 판독

실수 2 — "생성자의 loss가 계속 줄어야 정상"이라고 기대

GAN의 loss 곡선은 **등락** — D가 갱신될 때마다 G의 손실 함수 **자체**가 바뀌므로, "loss가 튀었다"가 반드시 실패를 뜻하지 않음. (반대로 EM의 우도처럼 **단조증가**하는 곡선을 기대하면 GAN의 "정상"을 "이상"으로 오독.) 실전에서는 **샘플 품질**을 직접 보고, D의 정확도가 50% 부근에서 **균형**을 유지하는지를 대시보드로.

실수 4 — "D의 정확도 50%"를 "잘 안 하고 있다"로 읽기

반대다 — 50%는 이 게임의 **목표** operating point(내쉬 균형). **정확도가 90%+로 치솟는 쪽**이 진짜 문제 신호 (위 "실전 학습"의 실수 1 참고).

확인 문제 (15.2)

1. **(손계산, Tier A)** 1차원 장난감 GAN에서 $\mu = 0.5$ 일 때 게임의 값 $V(D^*, G) = \log(1 + 1/\phi(0.5))$를 계산 ($\phi(0.5) = 0.3521$). 균형점 $\mu = 0$에서의 값 $V(D^*, G) = \log(1 + 1/\phi(0)) = \log(1 + 1/0.3989)$와 비교 — "$\mu = 0.5$에서는 아직 균형이 아니며, 생성자는 μ를 어떤 방향으로 움직일 유인이 있는가"를 한 문장으로 (힌트: $\mu = 0$에서의 값이 **더 작아야** $\mu = 0$이 G의 최소 — 즉 G의 관점의 내쉬 균형).
2. **(개념 서술)** 1차원 장난감에서 생성자는 결국 $\mu = 0$ **하나의 점**을 만듦 — 게임 관점에서는 "판별자를 완전히 속였다"의 **성공**이지만, 생성 품질 관점에서는 "다양성 0"의 **실패**. VAE의 ELBO에는 이 문제를 막는 **명시적인 항**이 있는데, 그 항이 무엇이며 GAN의 목적함수에는 왜 대응하는 항이 없는지를 두세 문장.

자주 묻는 질문: 모드 붕괴 · VAE/GAN/Diffusion

모드 붕괴는 왜 생기나요?

생성자는 판별자를 속이기만 하면 손실이 줄어듦 — 어떤 **한 종류**의 가짜가 판별자를 아주 잘 속인다는 것을 발견하면, 굳이 다른 다양한 가짜를 만들 이유가 없음. 판별자가 그 "치팅"을 알아채고 대응하기 전까지, 생성자는 그 **좁은 영역에만 몰릴** 유인. 근본 원인: VAE의 ELBO처럼 "**다양성을 명시적으로 요구하는 항**"이 GAN 기본 목적함수에는 **없음**.

Diffusion이 주류인데 VAE·GAN은 이제 안 쓰나요?

이미지 생성 분야에서는 Diffusion이 최근 주류가 됐지만, **VAE의 핵심 아이디어**(잠재 공간을 매끄럽게 만든다)는 여러 최신 시스템에서 Diffusion과 **결합**되어 사용 (예: 픽셀 공간이 아니라 **VAE로 압축한 잠재 공간**에서 Diffusion을 돌리는 구조 — Stable Diffusion이 정확히 이 방식). GAN도 **실시간성**이 중요한(반복 없이 한 번에 생성해야 하는) 응용에서는 여전히 사용 — "하나가 다른 셋을 완전히 대체했다"기보다, **각자 강점이 있는 도구로 함께**.

자주 묻는 질문: Diffusion 단계 수 · GAN 평가

Diffusion은 정말 T번의 단계를 다 돌리나요?

원래 DDPM은 $T = 1000$인데, **DDIM**(2020)이 역방향 과정을 거의 **결정적**으로 만들어 **약 50단계**로 줄이고, 이후 **증류**(distillation) 계열 기법(**LCM**, **progressive distillation** 등)으로 **10단계 이하** 가능. 대가 = "완전한 DDPM에 비하면 품질이 조금 떨어짐". "GAN보다 느리다"는 GAN 기준에서 **상대적** — 50단계 $\times$ 0.1초 $\approx$ 5초의 이미지 생성은 실제 응용에 **충분히 빠름**. (영상 생성처럼 샘플 수백 장을 만들어야 하는 분야에서는 이 차이가 더 중요.)

GAN은 ELBO 같은 "높을수록 좋은 단일 숫자"가 없는데, 학습 품질을 어떻게 평가?

GAN 계열은 단일 "우도" 숫자가 없어서 **샘플 기반** 지표가 따로 발전 — 대표적으로 **FID**(Fréchet Inception Distance): 실제 이미지와 생성 이미지를 InceptionNet 같은 **특징 추출기**로 변환한 뒤, 두 특징 분포의 **평균·공분산**(가우시안 근사) 사이의 **거리**. FID가 **작을수록** 생성 분포가 실제 분포에 **가깝**. VAE/GMM 계열은 "ELBO를 높인다"는 단일 숫자가 있어 비교가 쉬웠는데, GAN은 FID처럼 **외부 지표**가 필요 — 이(하나의 요인)가 이후 연구가 "**평가 가능한**" Diffusion 쪽으로 이동하는 데 일조.

연습문제 (코딩)

- **1.** `em_step`과 `kl_divergence_gaussian`(정규분포 $\mathcal{N}(\mu, \sigma^2)$와 표준정규분포 사이의 KL divergence) 완성. 핵심 줄: $D_{KL} = -0.5 \cdot (1 + \text{log_var} - \mu^2 - \exp(\text{log_var}))$.

연습문제 5: ELBO 유도 빈칸 폴백

자유 유도가 어려운 경우 — 빈칸을 채워라:

```
Step 1: log p(x) = log integral[ q(z|x) * (p(x,z) / q(z|x)) dz ]
                 = log E_q[ _____________ ]

Step 2: 옌센부등식 (가log 오목함수) 적용:
  log E_q[ p(x,z)/q(z|x) ] >= E_q[ log(_____________) ]

Step 3: p(x,z) = p(x|z) * p(z)로 분해하면:
  = E_q[ log p(x|z) ] + E_q[ _____________ ]

Step 4: E_q[ log p(z) - log q(z|x) ] = -D_KL(q(z|x) || p(z))결론

: log p(x) >= E_q[log p(x|z)] - D_KL(q(z|x) || p(z))   [ELBO]
```

정확성 확인

Step 2에서 **부등호가 등호가 아니라 부등식**인 이유를 한 문장으로 — (옌센 부등식이 **언제 등식**이 되는지: X가 **상수**일 때).

이번 주차 정리

1. **15.1 VAE** — **"EM의 E-step을 신경망이 맡는다"**. ELBO = **복원 항** $-$ **KL 정규화 항** (옌센 부등식 한 번으로 유도). 갭 $= D_{KL}(q\|p(z|x))$ — ELBO 최대화 $=$ 변분 베이즈. μ^2 **항**이 인코더 위치를 원점 쪽으로, σ^2 **항**이 너비를 제어. **생성 모드**는 인코더를 **버리고** 디코더만 통과. β-VAE가 "복원 품질 vs 잠재 공간 매끄러움"을 튜닝. "흐릿함"의 원인 = **기대값 최적화**.
2. **15.2 GAN** — min-max 게임의 **내쉬 균형** ($D(x) = 0.5, p_G = p_{\text{data}}$). 1차원 장난감에서 $V(D^*, G) = 2\,\text{JSD} - \log 4$로 **닫힌 형태**. 성공 신호 = V가 $-\log 4$ 근방에서 **안정**, $D^*(0) \to 0.5$. **모드 붕괴**의 원인 = 목적함수에 **다양성 항이 없음** (VAE에는 KL 항이 대응).
   WGAN/CycleGAN/StyleGAN/BigGAN = "**안정화**"가 3년의 주제.
3. **15.2 Diffusion + 비교** — 노이즈를 T단계로 **쪼갠 MSE 회귀**(가장 보통의 최적화 → **안정**). 역방향에도 $+\sigma_t z$(**다시 노이즈 추가**)가 **필요**. Stable Diffusion = **VAE 잠재 공간** $\times$ Diffusion. 세 원리: **잠재변수 · 학습 방법 · 안정성 · 생성 속도**로 비교 — 실전 시스템은 **조합**.

다음 주 — Week 16. Block B 캡스톤: 팀 프로젝트와 ML1 총정리

이번 장에서 배운 **세 원리(VAE · GAN · Diffusion)**가 팀 프로젝트의 **모델 선택** 도구 상자로 등장 — **안정성 · 생성 속도 · 품질**의 기준으로 선택하고 **정당화**.